Abjad AI at AraSeg Shared Task 2026: Auxiliary Loss for Arabic Sentence Segmentation

Mohammed Alsudairi **Naif Alharthi** **Taif Nono** **Faris Alasmary**
Anas Salamah **Shouq Sadah** **Lahouari Ghouti**
Abjad AI. Riyadh. Saudi Arabia
{malsudairi,nalharthi,tnono,falasmary,asalamah,ssadah,lghouti}@abjad.com.sa

Abstract

This paper describes the Abjad AI team's submission to the AraSeg 2026 Shared Task on Arabic sentence segmentation, covering the two punctuated tracks: Paragraph-Aware (PA) and No-Paragraph (NP). Rather than substituting a larger encoder, we retain the CAMeLBERT-MSA reference system and ask how much headroom remains in the task's formulation. Our system combines four components: boundary labels aligned to each word's final sub-token, an auxiliary classification head on the penultimate encoder layer trained jointly with the final-layer head, class-weighted cross-entropy for boundary sparsity, and overlap-averaged decoding that scores documents of arbitrary length without truncation. We ablate each component in isolation, including the two that the reference system already uses. Our solution achieves F_1 scores of 91.7 and 89.7 in the blind Test sets of the PA and NP sub-tasks, respectively.

1 Introduction

Sentence segmentation underpins nearly every downstream NLP pipeline, and errors made at this stage propagate through everything built on top. The problem is often treated as solved by punctuation heuristics, but this breaks down when boundaries must instead be recovered from syntactic and discourse cues, a setting in which token-classification models struggle: boundaries are sparse, documents routinely exceed encoder context windows, and models trained on punctuated text overfit to surface cues.

Arabic is exactly this setting: punctuation was adopted comparatively recently, is applied inconsistently across authors and genres (Yagi et al., 2024), sentences are frequently long and chained by coordinating conjunctions, and the script offers no capitalization cue at sentence onset. Yet the language has received little systematic attention. The AraSeg

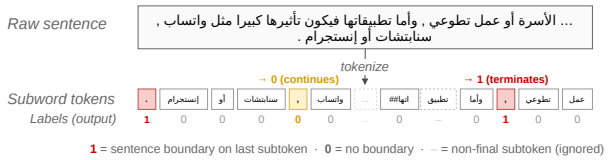


Figure 1: An example from the AraSeg Paragraph-Aware dataset where a comma (",") is marked as a sentence end, and another sentence where the same punctuation mark does not terminate a sentence.

2026 Shared Task (Elkholy et al., 2026b) addresses this gap with the first genre-diverse, manually annotated Arabic sentence segmentation corpus. We participate in two punctuated tracks: Paragraph-Aware (PA) and No-Paragraph (NP). Retaining punctuation does not reduce these tracks to punctuation detection as 39% of the corpus's 122k gold boundaries carry no sentence-final punctuation.

Our submission takes these constraints as fixed and asks how much headroom remains in the task's formulation rather than in encoder capacity, a reasonable place to look, since the benchmark reports saturating returns beyond 40% of the training data and lightweight encoders outperform LLMs under its hardest conditions. We retain the *CAMeLBERT-MSA* reference encoder (Inoue et al., 2021) and vary only how the task is presented to it (§4): final-sub-token label alignment and overlap-averaged decoding, both inherited unablated from the baseline, plus a penultimate-layer classification head and class-weighted cross-entropy, both ours. Keeping the checkpoint and training configuration fixed makes any score difference attributable to the formulation alone.

2 Background

The shared task formulates sentence segmentation as binary token classification: given a document $x = (x_1, \ldots, x_n)$, a system predicts for each token a label $y_i \in \{0, 1\}$ indicating whether a sentence

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1004–1009
October 28-29, 2026 ©2026 Association for Computational Linguistics

boundary follows; punctuation marks are themselves tokens and may or may not carry one. In NP, the document is a single undifferentiated token stream; in PA, paragraph boundaries are also given at input, and every paragraph boundary is also a sentence boundary. Figure 1 shows a shortened input-output example, illustrating a type of occurrence that make the task non-trivial: the same punctuation mark may or may not terminate a sentence.

3 Related Work

Prior approaches. Classical segmenters rely on punctuation-driven rules or statistical boundary cues (Kiss and Strunk, 2006; Sadvilkar and Neumann, 2020); recent multilingual models such as Ersatz (Wicks and Post, 2021), WtP (Minixhofer et al., 2023), and SaT (Frohmann et al., 2024) instead train transformer encoders to remain robust to noisy or absent punctuation via corruption of the training signal. For Arabic, prior work targets neighboring tasks: clause and discourse segmentation (Touir et al., 2008; Keskes et al., 2012), unpunctuated-text segmentation (Alshanqiti et al., 2022), and dialectal boundary detection (Mekki et al., 2022), but AraSeg (Elkholy et al., 2026b) is the first benchmark for MSA segmentation across genres under controlled conditions; its strongest baselines fine-tune CAMeLBERT (Inoue et al., 2021) per condition, adding a punctuation-dropout variant CAMeLBERT$_{\text{PnxDrop}}$ for robustness. Because AraSeg documents routinely exceed the 512-subword limit of BERT-style encoders, inference typically proceeds over overlapping windows (Devlin et al., 2019; Wang et al., 2019; Pappagari et al., 2019); since predictions near window edges rest on one-sided context, aggregating scores across overlapping windows rather than trusting a single window mitigates this (Ivgi et al., 2023).

Prediction depth. Probing studies show BERT's layers form a soft linguistic hierarchy: surface and syntactic phenomena peak at lower-to-middle depths, while final layers specialize toward the pretraining objective (Jawahar et al., 2019; Tenney et al., 2019). This has motivated reading from more than one depth, via learned layer mixing (Peters et al., 2018; Kondratyuk and Straka, 2019) or auxiliary classifiers at intermediate layers, deep supervision from computer vision (Lee et al., 2015; Szegedy et al., 2015), later used for early exiting in transformers (Xin et al., 2020). Boundary prediction is a largely local, syntactic decision, making readout depth a natural, previously unexamined question for segmentation.

Positioning. Our system (§4) starts from the strongest AraSeg baseline and departs from it in three ways: supervising classification heads at two depths and selecting the better on the Development set (henceforth "Dev"); training jointly on all four conditions (PA, NP, NoPnx-PA, NoPnx-NP), a robustness function like punctuation dropout but using the task's own parallel data rather than synthetic corruption; and reweighting the loss toward the sparse boundary class. Like the baseline, it aligns labels to the final sub-token and decodes via overlap-averaged windows.

4 System Overview

Our system keeps the encoder of the strongest AraSeg baseline, CAMeLBERT-MSA (Inoue et al., 2021), fine-tuned in full for binary token classification (§2), and changes only how the task is presented to it. A single model is trained jointly on all four punctuation and structure conditions and submitted to both tracks; the PA and NP inputs differ only in the paragraph-marker substitution of §4.2. The four components below are each ablated in isolation in Table 1; implementation details are in §5. We write $x_1, \ldots, x_n$ and $y_i \in \{0, 1\}$ for the input tokens and gold labels; the tokenizer splits each token into sub-tokens, $\tau(i)$ denotes the index of the *final* sub-token of x_i, and $h_t^{(\ell)}$ the hidden state at sub-token position t and encoder layer ℓ ($L = 12$ total).

4.1 Classification Heads at Two Depths

We replace the pretrained MLM head with two independent per-token linear classifiers, one over the final and and one over the penultimate layer, with dropout ($p = 0.1$) on their inputs. Each head reads the hidden states $h_{\tau(i)}^{(\ell)}$ ($\ell \in \{L-1, L\}$) and produces boundary probabilities $p_i^{(\ell)}$ via softmax. The two heads are trained jointly, with the total objective being the loss of Eq. (1), computed per head and summed. Following §3, we avoid committing to a readout depth a priori and instead supervise both, selecting on Dev. The penultimate head performed better, so all reported results use it; early stopping and checkpoint selection likewise monitor its Dev F_1. Averaging both heads' logits did not improve on the penultimate head alone.

4.2 Input Encoding

Final-sub-token label alignment. We place each label on its token's final sub-token, assigning the ignore index (-100) to all other sub-tokens, so supervised positions are exactly $\{\tau(i)\}_{i=1}^{n}$ contributing neither spurious loss nor duplicate votes. We prefer this to the more common first-sub-token convention because the boundary is a property of the token's *end*, and $\tau(i)$ is the last position within the token, after all attached clitics. The baseline uses the same convention without measuring it; Table 1 does.

Restoring the paragraph signal (PA). In PA, paragraph boundaries arrive as newline tokens that CAMeLBERT's tokenizer normalizes away as whitespace. Since this signal constitutes the only difference between PA and NP inputs, it would never reach the model. We replace each newline with the encoder's `[SEP]` token before tokenization, preserving token–label alignment (i.e., paragraph and sentence boundary). The marker is a maximally reliable cue once visible, identically at training and inference.

4.3 Overlap-Averaged Decoding

During training, documents are split into overlapping windows of 200 tokens. At inference, the document is tokenized once, without truncation, and split into overlapping windows of 512 sub-tokens. Each window is encoded independently. For a position covered by multiple windows, the selected head's *logits* are averaged uniformly over those windows.

$$\bar{z}_t \;=\; \frac{1}{|\mathcal{W}(t)|} \sum_{W \in \mathcal{W}(t)} z_t^{(W)},$$

where $\mathcal{W}(t)$ is the set of windows covering sub-token t and $z_t^{(W)}$ the logit vector in window W; argmax is taken after averaging, and $\hat{y}_i = \arg\max_k \bar{z}_{\tau(i),k}$ recovers one prediction per token via τ. The baseline decodes the same way with unreported parameters; Table 1 compares against disjoint windows and trusting each position's first window.

4.4 Training

Joint conditions. We train one model on the concatenated splits of all four conditions, unlike the baseline's model per condition. Since conditions present overlapping text with and without punctuation, the model repeatedly sees boundaries it must recover without punctuation support, the robustness function of CAMeLBERT-PnxDrop's synthetic corruption (§3), here from the task's own parallel data.

Class-weighted objective. Boundaries constitute roughly 8% of labeled positions (121,623 over ≈1.45M tokens; §5); word-based ones are precisely where the baseline is weakest (§1). We counter this imbalance by up-weighting the boundary class in the cross-entropy loss: non-boundary positions receive weight $c_0 = 1$ and boundary positions $c_1 = 14$. We also apply light label smoothing with $\epsilon = 0.01$, the true class receives target $\tilde{y}_{i,y_i} = 1 - \frac{\epsilon}{2}$ and the other class $\tilde{y}_{i,1-y_i} = \frac{\epsilon}{2}$. The per-head loss over the supervised positions $\mathcal{I} = \{\tau(i)\}$ is

$$\mathcal{L}^{(\ell)} = -\frac{1}{\sum_{i \in \mathcal{I}} c_{y_i}} \sum_{i \in \mathcal{I}} c_{y_i} \sum_{k \in \{0,1\}} \tilde{y}_{i,k} \, \log p_{i,k}^{(\ell)}. \tag{1}$$

Note the normalization by $\sum_{i \in \mathcal{I}} c_{y_i}$ rather than $|\mathcal{I}|$: boundary-heavy batches do not receive proportionally larger gradient steps. Because weighting trades precision for recall, Table 1 reports both alongside F_1 for $c_1 \in \{1, 5, 14\}$.

5 Experimental Setup

Data. We use only the official AraSeg releases, with no external data. The corpus comprises 2,187 documents from 30 sources across eight genres: 48,250 paragraphs, 121,623 sentences, and roughly 1.45M tokens, split into 1,703 documents (≈1.1M tokens) for Train, 222 (≈159K tokens) for Dev, and 262 (≈154K tokens) for Test (Elkholy et al., 2026a). Documents arrive pre-tokenized into words and sub-tokenized with the CAMeLBERT WordPiece tokenizer.

Implementation. The encoder is a fully fine-tuned *CAMeLBERT-MSA* (BERT-base: 12 layers, hidden size 768) (Inoue et al., 2021). Early stopping and checkpoint selection monitor the penultimate head's Dev set boundary F_1 (§4.1). The class weight, dropout, label-smoothing were all selected after a grid search. A class weight sample $c_1 \in \{1, 5, 14\}$ is shown in Table 1.

Evaluation. Systems are scored with boundary-level precision, recall, and F_1 under exact match against gold boundaries. We report all three metrics on both tracks. One caveat for reproducibility: checkpoint selection monitors a window-level micro-averaged boundary F_1 over Dev batches as a training-time proxy, whereas every score reported

	PA			NP		
	P	R	F_1	P	R	F_1
Full submitted system ($c_1 = 14$)	92.06	94.65	**93.00**	91.34	92.66	**91.56**
Classification head						
final-layer head	92.04	94.29	92.73	90.57	92.47	91.05
Label alignment						
first-sub-token convention	92.03	94.39	92.75	90.92	92.56	91.30
majority vote (all sub-tokens)	91.51	94.18	92.96	91.20	92.46	91.41
Decoding						
disjoint windows (non-overlapping)	91.53	**94.90**	92.83	89.50	93.60	91.16
single window per position (no averaging)	91.70	94.82	92.88	89.71	**93.68**	91.26
Training						
unweighted loss ($c_1 = 1$)	**94.06**	85.12	88.55	**92.93**	83.81	87.31
intermediate weight ($c_1 = 5$)	93.37	93.23	92.99	92.14	91.52	91.41

Table 1: Ablations on the PA and NP Dev sets. Each row below the first toggles exactly one component of the submitted system, holding all other components and the training configuration fixed. Boundary-level precision and recall; document-level macro-averaged F_1 (the official metric).

here uses the official evaluation script on full documents after overlap-averaged decoding.

6 Results

Table 1 reports ablation-based results using the Dev set. As reported, the submitted system reaches 93.00 and 91.56 F_1 scores in the PA and NP sub-tasks using the Dev set, respectively. This contrasts with the 91.7 and 89.7 scores achieved on the blind Test set, a drop of 1.3 points on PA and 1.9 on NP. Worth noting that, for the Dev set, the gap falls mainly on precision rather than recall.

Class weighting is the only component with a large effect. We compare $c_1 \in \{1, 5, 14\}$: removing weighting entirely ($c_1 = 1$) costs 4.45 F_1 on PA and 4.25 on NP, as the unweighted model underpredicts the sparse class. Beyond that, the exact weight barely matters: $c_1 = 5$ trails the submitted $c_1 = 14$ by only 0.01 F_1 on PA and 0.15 on NP. The gain comes from countering boundary sparsity in the first place. Reading from the penultimate layer improves over the final-layer head by 0.27 and 0.51 in the F_1 in the PA and NP sub-tasks, respectively.

Two conventions inherited from the reference system contribute little once measured in isolation: replacing final-sub-token alignment with the first-sub-token convention or majority voting over all sub-tokens costs at most 0.26 F_1 on either track, leaving the submitted choice marginally best on both. Decoding shows a similar pattern on PA, where disjoint windows and single-window decoding trail overlap-averaged decoding by at most 0.17, but on NP overlap averaging yields a somewhat larger benefit of 0.30–0.40 F_1. The alignment alternatives are defensible and nearly interchangeable defaults, plausibly within run-to-run variance (see Limitations); overlap-averaged decoding earns its place more clearly on NP. Headroom in the formulation lies almost entirely in the training objective and, secondarily, readout depth and decoding.

7 Conclusion

We held the AraSeg reference encoder fixed and asked how much headroom lies in the task's formulation. On the punctuated tracks, the answer is concentrated in one place: class-weighted cross-entropy against boundary sparsity accounts for over four points of F_1. In additional experiments beyond the reported ablations, a finer search over intermediate class weights yielded marginally higher Dev F_1 than the submitted setting, suggesting the weighting scheme is not yet fully optimized. A penultimate-layer readout adds a small, consistent gain, the inherited alignment conventions prove nearly interchangeable with their alternatives, and overlap-averaged decoding contributes a modest additional benefit on NP. A single model trained jointly on all four conditions and submitted to both tracks achieves 91.7 and 89.7 F_1 scores on the PA and NP sub-tasks in the Blind Test set, respectively. Natural next steps include the unpunctuated tracks, where sparsity is compounded by lost surface cues; a more systematic calibration of class weights, potentially tuned per track; threshold calibration as a lighter-weight alternative to loss reweighting; and probing readout depths below $L-1$.

Limitations

Our findings are tied to a single encoder and a single benchmark, and may not generalize across model capacities or scales. In addition, AraSeg boundaries are partly stylistic, so a single gold segmentation cannot credit valid alternative segmentations. Each configuration was trained with a single seed and we report no variance estimates: the class-weighting effect (over 4 F_1) is safely outside plausible noise, but the remaining ablation differences (0.01–0.51 F_1) are not. We therefore treat the exact-weight, alignment, and PA-decoding differences (≤ 0.26 F_1) as near-ties, and read the penultimate-layer and NP-decoding gains (0.27–0.51 F_1) as directionally consistent trends rather than established effects; the same caveat applies to the exploratory weight search noted in §7. Moreover, since we used the Dev set for both model selection and ablations, our Dev results are mildly optimistic, consistent with the 1.3–1.9-point drop on the Blind Test set. Because test labels are withheld, we cannot reproduce the punctuation- versus word-boundary analysis that motivated the system (§1). Finally, we participate only in the punctuated tracks even though our training uses NoPnx data.

References

Abdullah M. Alshanqiti, Sami Albouq, Ahmad B. Alkhodre, Abdallah Namoun, and Emad Nabil. 2022. Employing a multilingual transformer model for segmenting unpunctuated arabic text. *Applied Sciences*, 12(20).

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026a. Arabic sentence segmentation across genres and punctuation conditions. *Preprint*, arXiv:2606.08025.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026b. Arabic sentence segmentation shared task 2026. In *Proceedings of the Fourth Arabic Natural Language Processing Conference*, Budapest, Hungary. Association for Computational Linguistics.

Markus Frohmann, Igor Sterner, Ivan Vulić, Benjamin Minixhofer, and Markus Schedl. 2024. Segment any text: A universal approach for robust, efficient and adaptable sentence segmentation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11908–11941, Miami, Florida, USA. Association for Computational Linguistics.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Maor Ivgi, Uri Shaham, and Jonathan Berant. 2023. Efficient long-text understanding with short-text models. *Transactions of the Association for Computational Linguistics*, 11:284–299.

Ganesh Jawahar, Benoît Sagot, and Djamé Seddah. 2019. What does BERT learn about the structure of language? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3651–3657, Florence, Italy. Association for Computational Linguistics.

Iskandar Keskes, Farah Benamara, and Lamia Hadrich Belguith. 2012. Clause-based discourse segmentation of Arabic texts. In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12)*, pages 2826–2832, Istanbul, Turkey. European Language Resources Association (ELRA).

Tibor Kiss and Jan Strunk. 2006. Unsupervised multilingual sentence boundary detection. *Computational Linguistics*, 32(4):485–525.

Dan Kondratyuk and Milan Straka. 2019. 75 languages, 1 model: Parsing Universal Dependencies universally. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2779–2795, Hong Kong, China. Association for Computational Linguistics.

Chen-Yu Lee, Saining Xie, Patrick Gallagher, Zhengyou Zhang, and Zhuowen Tu. 2015. Deeply-Supervised Nets. In *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*, volume 38 of *Proceedings of Machine Learning Research*, pages 562–570, San Diego, California, USA. PMLR.

Asma Mekki, Inès Zribi, Mariem Ellouze, and Lamia Hadrich Belguith. 2022. Sentence boundary detection of various forms of tunisian arabic. *Language Resources and Evaluation*, 56(1):357–385.

Benjamin Minixhofer, Jonas Pfeiffer, and Ivan Vulić. 2023. Where's the point? self-supervised multilingual punctuation-agnostic sentence segmentation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1:*

Long Papers), pages 7215–7235, Toronto, Canada. Association for Computational Linguistics.

Raghavendra Pappagari, Piotr Zelasko, Jesús Villalba, Yishay Carmiel, and Najim Dehak. 2019. Hierarchical transformers for long document classification. In *2019 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pages 838–844.

Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana. Association for Computational Linguistics.

Nipun Sadvilkar and Mark Neumann. 2020. PySBD: Pragmatic sentence boundary disambiguation. In *Proceedings of Second Workshop for NLP Open Source Software (NLP-OSS)*, pages 110–114, Online. Association for Computational Linguistics.

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. Going deeper with convolutions. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9.

Ian Tenney, Dipanjan Das, and Ellie Pavlick. 2019. BERT rediscovers the classical NLP pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601, Florence, Italy. Association for Computational Linguistics.

Ameur A Touir, Hassan Mathkour, and Waleed Al-Sanea. 2008. Semantic-based segmentation of arabic texts. *Information Technology Journal*, 7(7).

Zhiguo Wang, Patrick Ng, Xiaofei Ma, Ramesh Nallapati, and Bing Xiang. 2019. Multi-passage BERT: A globally normalized BERT model for open-domain question answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 5878–5882, Hong Kong, China. Association for Computational Linguistics.

Rachel Wicks and Matt Post. 2021. A unified approach to sentence segmentation of punctuated text in many languages. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3995–4007, Online. Association for Computational Linguistics.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. 2020. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2246–2251, Online. Association for Computational Linguistics.

Sane Yagi, Shehdeh Fareh, Ashraf Elnagar, Mariam Balajeed, Abdalla El-mneizel, and Mohammad Al-Badawi. 2024. Is arabic punctuation rule-governed? *Cogent Arts & Humanities*, 11(1):2303818.